

PNP Formal Reconstruction Report

Compiled Lean theorem inventory and current claim boundary

Coordinate PNP-FORMAL-RECONSTRUCTION-STATUS-2026-07-10-12

The repository does not currently establish
 $P = NP$.

Public theorem emission is disabled. An audited concrete complexity target now exists, but the charged pipeline has not been linked to the raw machine kernel, its reviewed activation fingerprints are unset, and no root theorem exists. The abstract string-handle bridge is not a standard P-versus-NP formalization and is never an activation source.

Compiled declarations	2484
Theorem-kind declarations	883
Assumption-free theorem-kind declarations	793
Project-specific axioms	5
Concrete publication gate passed	false

Inventory SHA-256: 91b4db358afb1156eb39f3d97f49a5bb85a97f0b65ff1a879a51a6552ae15663

Generated from the pinned compiled Lean environment, not from source-text parsing.

Contents

1	Current authority and claim boundary	2
2	Concrete publication gate	2
2.1	Why the abstract bridge is ineligible	3
3	Formal milestone ledger	3
4	Compiled inventory summary	4
5	Remaining formal blockers	5
6	Historical report provenance	5
7	Verification	6

1 Current authority and claim boundary

This report is the current repository report surface. Its factual theorem inventory comes from the compiled PNP environment under `leanprover/lean4:v4.31.0`. The exporter enumerates `Environment.constants`, resolves each originating module, and calls `Lean.collectAxioms` for every exported project declaration. The committed inventory is `status/LEAN_THEOREM_INVENTORY.json`; the public mirror is byte-identical.

The target remains $P = NP$, but target wording—including the new inactive definition `PNP.Main.ConcretePEqualsNP`—is not theorem evidence. JavaScript acceptance, Boolean fields, JSON strings, historical release records, report prose, and external review cannot activate a theorem. Only the separately derived concrete publication gate may control theorem emission, and that gate is currently false.

Machine field	Current value
<code>mathematicalTheoremEstablished</code>	false
<code>publicTheoremEmissionAllowed</code>	false
<code>finalTheoremReady</code>	false
<code>publicTheoremStatement</code>	null
<code>rootLeanTheorem</code>	<code>PNP.Main.p_eq_np</code>
<code>rootLeanTheoremPresent</code>	false

2 Concrete publication gate

The compatibility entry is `PNP.Main.p_eq_np`; a matching name alone is insufficient. The separate publication target is `PNP.Main.ConcretePEqualsNP`. Eligibility also requires the exact theorem type, reviewed kernel fingerprints, and a tightly fixed Lean-standard axiom closure. The target is present as an inactive definition backed by finite charged-pipeline semantics; its presence is not a proof, and the compiler/refinement link to the raw machine kernel remains an explicit blocker.

The expected fingerprints are intentionally unset. This is a fail-closed migration gate, not a missing runtime input. A verifier must reject any attempt to set `passed=true` while a fingerprint is null, while the concrete model is ineligible, or while any other subcheck is false.

Gate subcheck	Result
<code>standardComplexityModelEligible</code>	false
<code>concreteTargetPresent</code>	true
<code>concreteTargetIsDefinition</code>	true
<code>concreteTargetKernelTypeFingerprintConfigured</code>	false
<code>concreteTargetKernelTypeFingerprintMatches</code>	false
<code>concreteTargetKernelValueFingerprintConfigured</code>	false
<code>concreteTargetKernelValueFingerprintMatches</code>	false
<code>compatibilityRootPresent</code>	false
<code>compatibilityRootIsTheorem</code>	false
<code>compatibilityRootHasExactConcreteType</code>	false
<code>compatibilityRootKernelTypeFingerprintConfigured</code>	false
<code>compatibilityRootKernelTypeFingerprintMatches</code>	false
<code>axiomClosureFingerprintConfigured</code>	false
<code>axiomClosureFingerprintMatches</code>	false
<code>sourceClosureFingerprintConfigured</code>	false
<code>sourceClosureFingerprintMatches</code>	false

Gate subcheck	Result
axiomClosureUsesOnlyLeanStandardAllowlist	false

Allowed foundation axioms are fixed to `Classical.choice`, `Quot.sound`, and `propext`. Project axioms, `sorryAx`, or any unknown axiom make the gate fail. The five currently disclosed project axioms are:

- `PNP.CheckPCCPackexp`
- `PNP.GeneratePCCPack`
- `PNP.LockedNANDThreshold`
- `PNP.ResidualBandExactMinimization`
- `PNP.SAT`

2.1 Why the abstract bridge is ineligible

The current `PNP.PEqualsNP` abbreviation compares witness classes whose language and algorithm structures carry string names or code handles rather than concrete execution semantics and proved polynomial bounds. Consequently, even a kernel-clean theorem of that abstract type would not establish the standard P-versus-NP statement. It is compatibility information only.

The separate `PNP.Concrete.PEqualsNP` target uses proof-bearing P and NP witnesses, canonical input/certificate pairing, finite program syntax, and charged polynomial runtime and output bounds. This is a substantive formalization milestone, but it is not yet proved equivalent to execution in the raw machine kernel. SAT completeness, SAT in P, and the root theorem also remain absent.

3 Formal milestone ledger

Each earned row requires exact compiled names and theorem kinds, empty axiom closures, per-name domain-separated kernel-type SHA-256 matches for 34 detailed candidates, and the pinned whole-Lean source closure. Human-readable scope remains conservative: type or source drift revokes credit.

Milestone	Status	Exact scope	Boundary
Concrete machine and cost kernel	formalized-foundation-only	Executable bitstrings and codecs, finite rule-list machine semantics, fuel-bounded execution, and proof-bearing natural-polynomial runtime witnesses.	This foundation alone does not define concrete P, NP, reductions, SAT, NP-completeness, or P = NP.
Concrete P, NP, and polynomial reductions	formalized-with-machine-link-pending	Bitstring languages, finite charged decision/verifier/function pipelines grounded in concrete machines, polynomial certificate/runtime/output bounds, many-one reductions, and the NP-complete-in-P implication.	The charged pipeline is not yet proved to compile or refine to one raw machine; concrete SAT and the publication root theorem remain absent.

Milestone	Status	Exact scope	Boundary
Typed direct-wire NAND semantics	formalized	Typed topological Boolean NAND programs and ordered multi-output direct-wire semantics.	This does not establish circuit minimization, SAT, or $P = NP$.
Finite enumeration, equivalence, and reference minimum	formalized	Exhaustive finite Boolean direct-wire search under the empty-profile reference model.	No polynomial-runtime result is obtained from this exhaustive reference search.
Concrete framed replacement and slack	formalized	The concrete serial framed context with support outputs and explicit bypass wires.	This is not an arbitrary-support replacement theorem for the report's global family.
Locked-NAND local candidates and baseline accounting	formalized	Typed local macro candidates, source-derived counts, and five finite local square baselines.	Local minima are not a global BaselineDistinct or locked-NAND threshold theorem.
Conditional locked-NAND threshold boundary	formalized-with-premises	A proof-bearing six-premise candidate boundary for an arbitrary satisfiable proposition.	The premises are not instantiated by a uniform SAT-to-locked-NAND builder.
Fail-closed explicit-list residual routes	formalized-explicit-list-only	Executable strict-gain search over a caller-supplied finite implementation list.	Unresolved excludes no gain outside the supplied list and cannot imply ZeroSlack.
Global locked-NAND construction and threshold	not-formalized	A uniform answer-independent SAT instance builder, global baseline, trace equivalence, and threshold.	The conditional boundary does not discharge this missing theorem.
Global ZeroSlack, PCCMin, and polynomial runtime	not-formalized	Complete residual routing, global ZeroSlack contradiction, exact minimization, and polynomial bounds.	Proof-bearing result structures and explicit-list failure do not supply global completeness.
Concrete standard P-versus-NP target and root theorem	not-formalized	Raw-machine-linked complexity classes, concrete SAT completeness and a SAT decider, plus the publication root theorem.	The concrete target definition is inactive: raw-machine linkage, concrete SAT completeness/deciding, and <code>PNP.Main.p_eq_np</code> remain absent; the legacy string-handle bridge is ineligible.

4 Compiled inventory summary

The inventory contains 2484 exported `PNP.*` kernel declarations from 26 modules. It excludes 36 private compiler auxiliaries. Counts describe the compiled environment; they do not measure mathematical completeness.

Module	Declarations	Theorems
PNP.Bridge	93	22
PNP.Complexity	93	20
PNP.Concrete.BitString	123	60
PNP.Concrete.Complexity	314	93
PNP.Concrete.Machine	284	67

Module	Declarations	Theorems
PNP.Concrete.Target	2	1
PNP.DirectWireBaseline	48	25
PNP.LockedNAND	11	4
PNP.LockedNANDBaseline	57	22
PNP.LockedNANDDirect	84	57
PNP.LockedNANDLocalBaseline	30	22
PNP.LockedNANDMacros	288	98
PNP.LockedNANDPrefix	84	40
PNP.LockedNANDThresholdBoundary	60	35
PNP.Main	36	12
PNP.NANDComposition	77	38
PNP.NANDEnumerator	120	46
PNP.NANDMinimum	56	23
PNP.NANDSemantics	198	80
PNP.NANDSlack	15	12
PNP.NANDTruthTable	72	28
PNP.PCCMin	44	8
PNP.ResidualBand	9	2
PNP.ResidualRoutes	141	46
PNP.SAT	4	1
PNP.ZeroSlack	141	21

5 Remaining formal blockers

Seven formal blockers remain active:

1. `Formal.ConcreteComplexityMachineLink`
2. `Formal.ConcreteSAT`
3. `Formal.LockedNANDThreshold`
4. `Formal.ResidualBandMinimizer`
5. `Formal.ZeroSlack`
6. `Formal.PolynomialRuntimeAndCertificateBounds`
7. `Formal.RootTheoremAndAxiomAudit`

The conditional locked-NAND threshold module assumes typed baseline and full candidates, baseline conditions, preservation of existing outputs, an unsatisfiable final-zero law, and satisfiable final-output laws. The explicit residual scanner searches only a caller-supplied finite list. Neither result constructs the missing global reduction or proves global ZeroSlack, polynomial PCCMin, SAT in P, or P equals NP.

6 Historical report provenance

Earlier 56-page report revisions stated a direct checker-mediated P-versus-NP claim. Those bytes remain historical audit material at their pinned legacy coordinates and through the explicit archive and supersession records. They are not imported into, quoted as authority by, or used to generate this report. File identity and replay of historical predicates do not establish the named mathematical propositions.

Source tag `final-pnp-proof-report-hardened-7072f8d`
Source commit `7072f8d0bda6d44d240f9bb3fad624fd357e1278`
Archive manifest `archive/legacy-v0/ARCHIVE.json`

7 Verification

The current reconstruction can be checked with:

```
lake build PNP
node scripts/export-lean-theorem-inventory.mjs --check
node scripts/generate-formal-publication.mjs --check
node pcc-formal-reconstruction-status0.mjs --json
npm run pnp:verify -- --no-write
npm run report:check
```

The inventory coordinate is PNP-LEAN-THEOREM-INVENTORY-2026-07-10-12. Its exact byte digest is 91b4db358afb1156eb39f3d97f49a5bb85a97f0b65ff1a879a51a6552ae15663. The report is regenerated from that inventory and the reviewed publication map. The reviewed milestone source-closure SHA-256 is 0b48d081b1b9ab050184c895c7e1b99ed118c56c9737c7a76a5580dc1bf001be; the computed and pinned values match. Successful regeneration confirms consistency of these status surfaces; it does not turn an absent concrete theorem into a proof.